

Transformer 把过去序列模型里依赖 RNN / CNN 的“按顺序处理文本”的方式，改成了完全基于 Attention 的并行处理方式，从而让模型更快、更容易捕捉长距离关系，并成为后来大语言模型的基础架构。

在 Transformer 之前，机器翻译、语言建模等任务主要依赖 **RNN / LSTM / GRU** 这类循环神经网络。

它们的问题是：**必须一个词一个词地顺序处理**。

比如句子：

| The animal didn't cross the street because it was too tired.

模型要理解 `it` 指的是 `animal`，就需要从前面一路传递信息到后面。序列越长，信息传递越困难，训练也越慢。

论文认为，RNN 的根本瓶颈在于：

顺序计算太强，难以并行。

所以作者提出：

| 不要再让模型按顺序“读句子”，而是**让每个词直接和所有词建立关系**。

这就是 Transformer 的核心思想。论文摘要中明确说，Transformer 完全基于 attention mechanism，不再使用 recurrence 和 convolution。

通俗理解：

以前模型像人从左到右逐字读；Transformer 像把整句话摊开，让每个词同时观察其他所有词。

2. Transformer 的整体结构：Encoder-Decoder

Transformer 仍然采用经典的 **Encoder-Decoder** 结构。

Encoder：负责理解输入

比如输入英文句子：

| I love machine learning.

Encoder 的任务是把每个词转成一组带上下文含义的向量。

不是简单地理解：

- I = 我
- love = 爱
- machine = 机器
- learning = 学习

而是理解整句话中的关系：

- machine learning 是一个整体概念；
- love 的对象是 machine learning ；
- I 是动作发出者。

论文中 Transformer 的 Encoder 由 **6 层相同结构堆叠而成**，每层包含两个核心模块：Multi-Head Self-Attention 和 Feed-Forward Network。

Decoder：负责生成输出

比如翻译成中文：

| 我喜欢机器学习。

Decoder 的任务是一步一步生成目标语言。

它每生成一个词，都要参考两类信息：

1. 已经生成的中文词；
2. Encoder 对英文原句的理解结果。

论文中的 Decoder 也是 **6 层堆叠**，但比 Encoder 多一个模块：它不仅要看自己已经生成的内容，还要看 Encoder 的输出。

通俗理解：

Encoder 是“读懂原文”，Decoder 是“根据原文和已写内容继续往下写”。

3. Attention 是什么？

论文把 Attention 描述为：给定一个 Query 和一组 Key-Value，输出是 Value 的加权求和，权重由 Query 和 Key 的匹配程度决定。

这句话比较抽象，可以这样理解。

假设你在读一句话：

The cat sat on the mat because it was tired.

当模型处理 `it` 时，它要判断 `it` 到底指谁。

于是 `it` 会去“询问”句子里其他词：

- `cat`：和我关系大吗？
- `mat`：和我关系大吗？
- `tired`：和我关系大吗？

最后模型发现：

`it` 和 `cat` 的关系更强，所以给 `cat` 更高权重。

这就是 Attention：

当前词根据需要，主动去关注句子里最相关的其他词。

4. Query、Key、Value 怎么理解？

Attention 里的三个核心概念是：

概念	通俗解释
Query	我现在想找什么信息
Key	每个词提供的“索引标签”
Value	每个词真正携带的信息

可以用搜索引擎类比：

你输入搜索词，这是 **Query** 。

网页有标题、关键词、标签，这是 **Key** 。

网页正文内容，就是 **Value** 。

模型会用 Query 和 Key 计算匹配度，然后根据匹配度从 Value 里提取信息。

所以 Attention 不是简单地平均看所有词，而是：

谁更相关，就多看谁。

5. Scaled Dot-Product Attention：核心公式

论文提出的基础注意力机制叫 **Scaled Dot-Product Attention**：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

这个公式可以拆成三步：

1. **Q 和 K 做点积**：判断当前词和其他词的相关性；
2. **除以 $\sqrt{d_k}$** ：防止数值太大，避免 softmax 过度极端；
3. **softmax 后乘以 V**：把相关词的信息按权重汇总回来。

通俗理解：

先打分，再归一化成权重，最后按权重汇总信息。

为什么要除以 $\sqrt{d_k}$ ？

因为当向量维度很高时，点积结果可能变得很大，softmax 会变得非常尖锐，**导致梯度很小**，训练困难。****论文专门指出**，用缩放项可以缓解这个问题。

6. Multi-Head Attention：为什么要“多头”？

如果只有一个 Attention Head，模型只能从一个角度看句子。

但一句话里的关系有很多种：

- 主谓关系；
- 指代关系；
- 修饰关系；
- 位置关系；
- 语义关系。

所以 Transformer 使用 **Multi-Head Attention**。

论文中的 base model 使用 **8 个 attention heads**，每个 head 关注不同的表示子空间，然后把结果拼接起来。

通俗理解：

一个 head 像一个专家。多个 head 就像多个专家同时看同一句话：有人看语法，有人看指代，有人看语义，有人看位置。最后把大家的判断合并。

这就是为什么 Multi-Head Attention 比单头 Attention 更强。

7. Self-Attention: Transformer 的真正核心

Self-Attention 的意思是：

一个序列内部的词彼此看彼此。

比如句子：

The bank raised interest rates.

模型处理 bank 时，会关注：

- raised
- interest
- rates

从而判断这里的 bank 更可能是“银行”，而不是“河岸”。

Self-Attention 的优势是：

每个词都可以直接看到所有其他词。

这和 RNN 不同。

RNN 里，第一个词的信息要传到最后一个词，需要经过很多步。

Self-Attention 里，第一个词和最后一个词可以一步建立联系。

论文也在 Why Self-Attention 部分强调，自注意力层连接任意两个位置只需要常数级路径长度，而 RNN 需要随着序列长度增加。

通俗理解：

RNN 是“传话游戏”，信息一站一站传；Self-Attention 是“群聊”，所有词可以直接互相发消息。

8. Masked Self-Attention: 为什么 Decoder 不能偷看未来？

在生成文本时，模型不能提前看到答案。

比如要生成：

当模型生成“喜欢”时，它只能看到前面的“我”，不能看到后面的“机器 学习”。

否则训练时它就相当于作弊。

所以 Decoder 里的 Self-Attention 要加 **Mask** 。

Mask 的作用是：

遮住当前位置之后的词，让模型只能看过去，不能看未来。

论文指出，Decoder 的 self-attention 会阻止当前位置关注后续位置，从而保证第 i 个位置的预测只能依赖 i 之前的已知输出。

通俗理解：

训练时把答案盖住，让模型学会真正往下预测，而不是抄答案。

9. Positional Encoding：没有 RNN，模型怎么知道词序？

Transformer 不用 RNN，也不用 CNN，所以它天然不知道词的顺序。

但语言顺序非常重要。

比如：

dog bites man
man bites dog

词一样，但顺序不同，意思完全不同。

所以 Transformer 要加入 **Positional Encoding**，也就是“位置编码”。

论文使用的是正弦和余弦函数生成的位置编码，并把它加到词向量上。

通俗理解：

词向量告诉模型“这个词是什么”，位置编码告诉模型“这个词在第几个位置”。

就像给每个词贴一个坐标：

- 第 1 个词；
- 第 2 个词；
- 第 3 个词；

- 第 4 个词。

这样模型才能区分词序。

10. Feed-Forward Network: Attention 之后还要“加工信息”

Transformer 每一层除了 Attention，还有一个 **Position-wise Feed-Forward Network**。

论文中这个模块由两层线性变换和中间的 ReLU 激活组成，输入输出维度是 512，中间层维度是 2048。

它的作用可以理解为：

Attention 负责“从哪里拿信息”；

Feed-Forward 负责“拿到信息后怎么加工”。

比如 Attention 帮模型发现：

it 和 cat 关系很强。

Feed-Forward 接着把这个信息转成更复杂、更抽象的语义表示。

通俗理解：

Attention 像检索相关信息，**Feed-Forward** 像对检索到的信息做进一步思考和加工。

11. Add & Norm: 为什么有残差连接和 LayerNorm?

每个子层外面都有：

- Residual Connection;
- Layer Normalization。

也就是论文图里的 **Add & Norm**。

它的作用是让深层网络更稳定、更容易训练。

Residual Connection 可以理解为：

不要完全依赖新计算结果，也保留原来的信息。

LayerNorm 可以理解为：

把数值整理到稳定范围里，避免训练过程忽大忽小。

通俗理解：

Residual 是“别把原始信息弄丢”，**LayerNorm** 是“让每一层输出更稳定”。

12. 为什么 Self-Attention 比 RNN / CNN 更有优势？

论文比较了 Self-Attention、RNN、CNN 的三个指标：

- 1. 每层计算复杂度；
- 2. 能否并行；
- 3. 长距离依赖路径长度。

核心结论是：

模型	问题
RNN	必须顺序计算，不好并行，长距离依赖难
CNN	可以并行，但要堆很多层才能看远距离
Self-Attention	每个词可以直接看所有词，并行能力强，长距离依赖路径短

论文表 1 中指出，Self-Attention 的 sequential operations 是 $O(1)$ ，而 Recurrent 是 $O(n)$ 。也就是说，自注意力可以高度并行，而 RNN 需要按序列长度逐步计算。

通俗理解：

RNN 像排队过安检，一个一个来；**Transformer** 像多个安检口同时开放，大家可以并行处理。

13. 训练设置和实验结果

论文主要在机器翻译任务上验证 Transformer。

英德翻译

Transformer big 在 WMT 2014 English-to-German 任务上取得 **28.4 BLEU**，比当时已有最佳结果高出 2 个 BLEU 以上。

英法翻译

Transformer big 在 WMT 2014 English-to-French 上取得 **41.8 BLEU**，并且训练成本显著低于之前的强模型。

训练速度

论文指出 base model 在 8 块 P100 GPU 上训练约 12 小时，big model 训练约 3.5 天。

这个结果当时非常关键：

Transformer 不仅效果好，而且训练更快、成本更低。

14. 论文为什么重要？

这篇论文的重要性不只是提出了一个翻译模型，而是开创了一个新的架构范式。

它证明：

处理序列任务，不一定需要 RNN，也不一定需要 CNN。Attention 本身就足够强。

这也是标题 **Attention Is All You Need** 的含义。

它后来成为：

- BERT；
- **GPT**；
- T5；
- LLaMA；
- **Claude**；
- **Gemini**；

等大模型的基础。

通俗地说：

这篇论文相当于现代大语言模型的“架构原点”。

15. 用一句大白话总结 Transformer

Transformer 的工作方式可以这样理解：

把一句话里的所有词同时摊开，让每个词根据任务需要去关注其他词；多个注意力头从不同角度理解关系；再加上位置编码告诉模型词序；最后通过多层堆叠，把词和词之间的复杂关系加工成

可用于翻译、生成和理解的表示。

16. 最适合记住的核心框架

你可以按这个流程理解 Transformer：

```
输入文本
↓
词向量 Embedding: 把词变成数字向量
↓
位置编码 Positional Encoding: 告诉模型词的顺序
↓
Self-Attention: 每个词看其他所有词
↓
Multi-Head Attention: 多个角度看关系
↓
Feed-Forward: 加工每个位置的信息
↓
Add & Norm: 稳定训练, 保留原信息
↓
Encoder 表示 / Decoder 生成
↓
输出下一个词的概率
```

更简单地说：

```
词是什么 + 词在哪里 + 词和谁有关
= Transformer 对语言的理解
```

17. 对你学习 AI 工程最关键的启发

这篇论文对工程实践有三个特别重要的启发：

第一，**架构创新往往来自对瓶颈的重新定义**。作者不是单纯改进 RNN，而是直接问：为什么一定要顺序处理？

第二，**Attention 本质上是一种可学习的信息路由机制**。模型不是固定读信息，而是动态决定“该看谁、看多少”。

第三，**Transformer 的成功不只是算法效果，而是工程友好**。它可以并行训练，适合 GPU 矩阵计算，所以能在大规模数据和大模型时代迅速放大。

这也是为什么它后来能支撑 GPT 这类大语言模型。